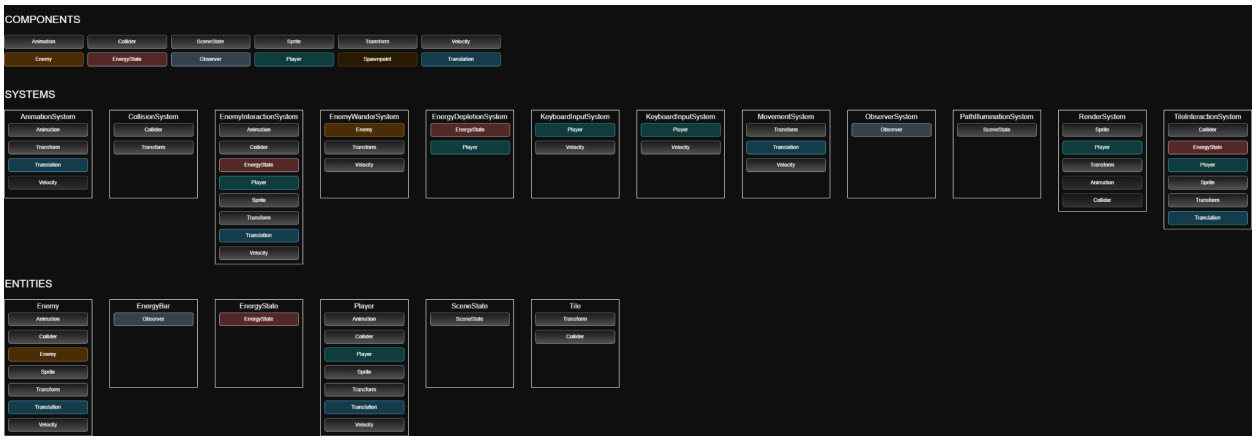
Overview

Name: Kate
Github handle: katelylynn
Github repo: https://github.com/katelylynn/maziac

No planning project tools because I'm working individually. I have a README in the Github repo for tracking my progress on this project (under the "docs" folder).

Engine Design



Using lecture material? Yes.

2D or 3D? 2D using SDL.

Pure or hybrid? Pure ECS because I've made object oriented games before and I want to get more comfortable with ECS architecture.

Systems

From lecture: Animation, Collision, KeyboardInput, Render
New / updated:

- EnemyInteraction - Handles the interactions between enemies and the player. Output of the fight depends on the amount of energy the player has and whether they have a weapon. Acts like the “EventResponseSystem” from the lecture code but is specialized for enemy/player interactions.
- EnemyWander - Handles random movement behavior for each enemy. Prevents the enemy from wandering too far from their initial start point.
- EnergyDepletion - Handles energy depletion and calling “game over” when the player has no energy left.
- Movement - Handles each characters’ discrete tile movement using Translation, Transform and Velocity. Differs from the lecture code because character movement snaps to each tile, versus being continuous.
- Observer - Executes behavior for an entity if a value that entity is watching changes.
- PathIllumination - Changes the tiles in the specified path to be "highlighted" until the path's life (time) runs out.
- TileInteraction - Handles the interactions (collisions) between the player and each of the tile types. Acts like the “EventResponseSystem” from the lecture code but is specialized for tile/player interactions.

Removed:

- Camera - Map is stretched to fit the screen view and the player moves around the screen space.

See engine design diagram for the components each system checks for.

Game Design

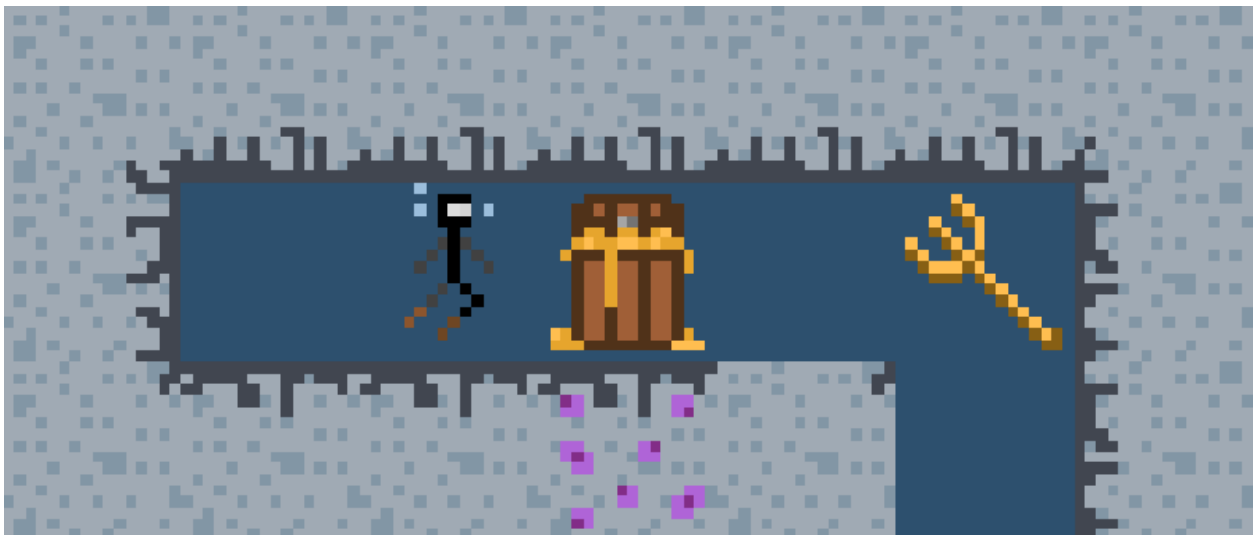
Game name: MAZIAC

Description: Based on the ZX Spectrum game “Maziacs”, Maziac is an underwater arcade game where you explore a maze and fight off enemies to find treasure.

Narrative: You’ve heard rumors of an abandoned treasure buried somewhere in the deepest part of the ocean. You take a boat and dive into a ravine where the treasure is allegedly located. A shimmer catches your eye, and as you squeeze through a crack in the ravine wall, your tank gets stuck and you pass out from lack of oxygen. You expect to die, but you wake up breathing because of some shiny pink algae on the walls. Using this discovery, you make your way through the tunnels at the bottom of the ocean, searching for the lost treasure and facing the monsters that guard it.

Platforms: Windows and Web.

Art assets: Made by me using Aseprite (see screenshot below).



High level features:

- Maze🧩- contains item tiles, a treasure tile and an exit tile.
- Guide tiles ❓- tiles that, when activated, highlights a path towards the treasure.
- Energy tiles ⏳ - drains over time. When it reaches zero, you die.
- Item tiles🎲- tiles that give you weapons or the treasure.
- Enemies🦑 - fighting them drains your energy.

Tools and Risks

Tools: CLion (IDE), Aseprite (Pixel art), Tiled (Map creation), and TinyXML (XML parsing).

Risk: My scope may be too big given that I have other school courses, minimal free time and less C++ experience.

Mitigation plan:
I am willing to do the following to reduce the scope:

- Opt for a single handmade map instead of procedurally generated maps,
- Produce lower resolution sprites (such as 16x16 instead of 64x64), and/or
- Exclude sound effects.

Along with reducing the scope, I will focus on developing the core game loop before any “nice-to-have” features.

Post-project reflection:
I ended up reducing the scope by all three points mentioned above. I excluded procedural generation and sound effects, and kept sprites to size 16x16. This helped me achieve a solid game loop by the deadline, while still balancing my other life priorities.